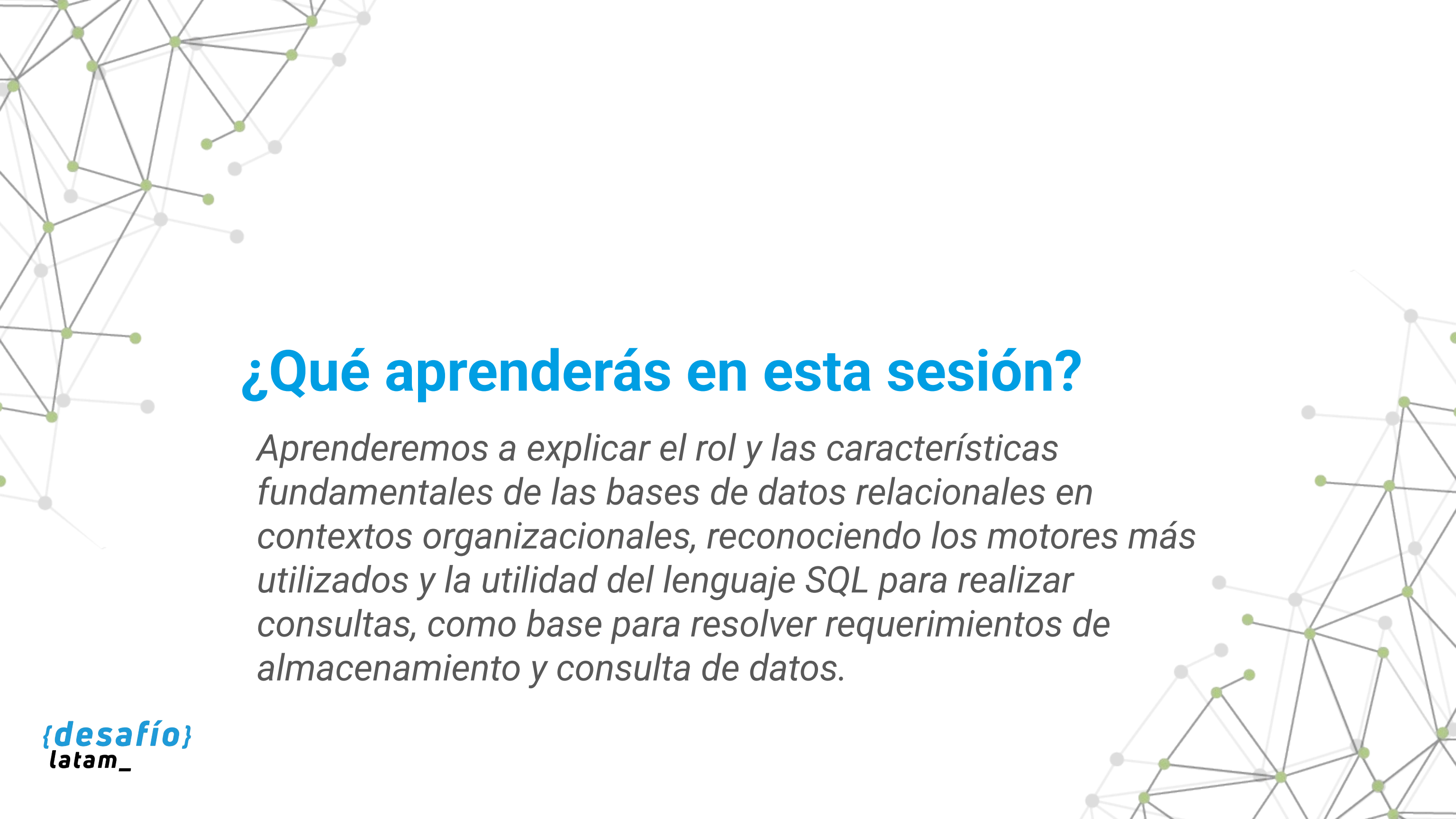




Fundamentos de Bases de Datos y Consultas Básicas

Clase 1 - Introducción a las bases de datos relacionales



¿Qué aprenderás en esta sesión?

Aprenderemos a explicar el rol y las características fundamentales de las bases de datos relacionales en contextos organizacionales, reconociendo los motores más utilizados y la utilidad del lenguaje SQL para realizar consultas, como base para resolver requerimientos de almacenamiento y consulta de datos.

Desafío inicial

Estás iniciando como analista de datos en una empresa de logística que trabaja con grandes volúmenes de información. Tu jefatura te asigna revisar y documentar el sistema actual de gestión de datos, ya que deben implementar un sistema de reportes automáticos para la toma de decisiones.

Actualmente, la información se almacena en diferentes hojas de cálculo distribuidas.

La empresa no tiene claridad sobre dónde está la información crítica, ni cómo relacionarla entre áreas. Los datos se actualizan manualmente, no hay integridad entre ellos, y esto genera retrasos y errores en los informes de gestión.

Tu tarea consiste en presentar a tu equipo una solución concreta para reemplazar este sistema de hojas sueltas por una arquitectura de datos centralizada, automatizada y escalable. Para ello deberás explicar qué son las bases de datos relacionales, por qué son necesarias, cómo se implementan, qué herramientas existen, y por qué el lenguaje SQL es clave en este proceso.

El rol de las bases de datos relacionales en la organización

¿Por qué las organizaciones prefieren usar bases de datos relacionales en lugar de hojas de cálculo o archivos planos?

Una base de datos relacional es un sistema estructurado para almacenar información en tablas que se pueden vincular entre sí. A diferencia de los archivos planos o hojas de cálculo, que presentan limitaciones en escalabilidad, integridad y seguridad, las bases de datos relacionales permiten definir relaciones, aplicar reglas de integridad y ejecutar consultas complejas de manera eficiente. Este enfoque estructurado es fundamental para garantizar que la información sea precisa, actualizada y accesible en contextos colaborativos y de alta demanda.

Las bases de datos relacionales se basan en el modelo relacional propuesto por Edgar F. Codd (1970), donde los datos se almacenan en tablas bidimensionales con filas (tuplas) y columnas (atributos). Estas tablas pueden relacionarse entre sí mediante llaves primarias y foráneas, permitiendo organizar datos complejos de forma modular y eficiente.

Aplicación en contextos productivos

En sectores como:



Logística

Relación entre pedidos, productos, bodegas y transportistas.



Salud

Historiales de pacientes relacionados con citas, médicos y tratamientos.



Educación

Registro de matrículas, estudiantes, cursos y calificaciones.



Retail

Relación entre clientes, órdenes, productos, pagos y despachos.

Fundamentos del modelo relacional y normalización de datos

Modelo entidad-relación

Es una metodología de modelado que define las entidades (como clientes, productos o órdenes), sus atributos y las relaciones entre ellas. Por ejemplo, una entidad "Cliente" puede tener atributos como nombre, correo y ciudad; y una relación "Compra" puede vincular clientes con productos.



Normalización

Es un proceso sistemático para organizar los datos que:

- Evita redundancia.
- Garantiza consistencia.
- Facilita mantenimiento.

Una base normalizada está dividida en múltiples tablas especializadas, lo que permite modificar información en un solo lugar (por ejemplo, cambiar el correo de un cliente sin afectar otras tablas).

Tabla clientes	Tabla productos	Tabla órdenes
id_cliente	id_producto	id_orden
nombre	nombre	id_cliente
ciudad	precio	id_producto
correo	stock	fecha_orden

Tabla 1. Ejemplo de estructura relacional normalizada

Fuente: Desafío Latam

Ejemplo:

```
SELECT nombre
FROM clientes c
JOIN ordenes o
ON c.id_cliente = o.id_cliente
WHERE o.fecha_orden
BETWEEN '2024-07-01'
AND '2024-07-31';
```


Buenas prácticas en el diseño relacional

1

Definir llaves primarias claras

Cada tabla debe tener un identificador único que permita distinguir registros (por ejemplo, id_cliente).

2

Usar llaves foráneas correctamente

Estas permiten vincular registros entre tablas, asegurando integridad referencial.

3

Evitar redundancias

Aplicando normalización para separar entidades que pueden cambiar de forma independiente.

4

Documentar el modelo de datos

Facilita su comprensión por equipos técnicos y no técnicos.

5

Planificar escalabilidad

Diseñar pensando en el crecimiento del volumen de datos.

Errores comunes

Diseño "monolítico" en una sola tabla

Lo que genera datos repetidos y dificulta el mantenimiento.

Ausencia de restricciones de integridad

Como llaves primarias o foráneas, lo que permite errores y datos huérfanos.

Nombres poco descriptivos en columnas o tablas

Dificulta el uso y la comprensión del modelo.

Modificaciones directas sin respaldo

Pueden generar pérdidas de datos si no se aplican buenas prácticas de auditoría.

No considerar relaciones m:n (muchos a muchos)

Que requieren tablas intermedias para mantenerse consistentes.

Entender el papel de las bases de datos relacionales en una organización permite construir sistemas escalables, coherentes y auditables. Una vez comprendida su relevancia, debemos conocer las herramientas que permiten implementarlas de manera efectiva: los sistemas gestores de bases de datos.

Características de un RDBMS y motores más utilizados

¿Qué hace que un sistema gestor de base de datos (RDBMS) sea confiable y eficaz?

Un Sistema Gestor de Base de Datos Relacional (RDBMS) es un software especializado en crear, mantener, consultar y proteger bases de datos relacionales. Estos sistemas gestionan las estructuras lógicas (tablas, vistas, índices) y físicas (archivos en disco) necesarias para garantizar operaciones seguras y eficientes, incluso ante el acceso concurrente de múltiples usuarios.

Entre sus funcionalidades esenciales destacan:



Gestor de transacciones que asegura coherencia de los datos incluso ante fallos.



Motor de almacenamiento optimizado para lectura y escritura eficiente.



Lenguaje SQL como interfaz estándar para consultas y mantenimiento.



Sistema de permisos que otorga control de acceso granular a usuarios y roles.

Características técnicas clave de un RDBMS

Transaccionalidad (modelo ACID)

- Atomicidad: Las operaciones se completan en su totalidad o no se ejecutan.
- Consistencia: Cada transacción deja la base en un estado válido.
- Aislamiento: Las operaciones simultáneas no se afectan entre sí.
- Durabilidad: Los cambios exitosos persisten ante fallos.

Escalabilidad

- Vertical: Incrementar recursos del servidor (CPU, RAM).
- Horizontal: Distribuir datos entre múltiples nodos (sharding, replicación).

Sistema de permisos

- Roles personalizados: Lectura, escritura, administración.
- Seguridad por usuario: Previene accesos no autorizados.

Comparativa de motores RDBMS

Motor	Casos de uso reales	Público objetivo	Tipo de licencia
MySQL	Sitios web, startups, aplicaciones	Empresas pequeñas-medias	Open Source
	SaaS		
PostgreSQL	BI, ciencia de datos, gobiernos	Instituciones públicas	Open Source
SQL Server	ERP, CRM corporativo	Empresas Microsoft	Comercial
Oracle	Banca, salud, defensa	Grandes corporaciones	Comercial
SQLite	Apps Android, IoT	Desarrollo embebido	Open Source

Tabla 2. Principales motores de bases de datos relacionales y sus aplicaciones

Fuente: Desafío Latam

Cada motor ofrece ventajas específicas en términos de rendimiento, extensibilidad, licenciamiento y comunidad.

Por ejemplo, PostgreSQL permite trabajar con funciones geoespaciales, mientras que MySQL destaca por su ligereza.

Buenas prácticas al elegir e implementar un RDBMS



Evaluar volumen y tipo de datos

Elegir motores que soporten el volumen actual y proyectado.



Validar integraciones necesarias

Algunos motores tienen mejores conectores con lenguajes específicos (como Python o R).



Revisar compatibilidad con el sistema operativo

Algunos funcionan mejor en entornos Linux, otros en Windows.



Asegurar soporte y comunidad activa

Facilita la solución de errores y la actualización.



Planificar estrategia de backup y recuperación

Cada motor ofrece diferentes herramientas nativas.

Errores comunes

Usar SQLite en ambientes multiusuario

Este motor es ligero pero no apto para concurrencia alta.

Ignorar las diferencias entre versiones

Algunas funciones SQL varían entre motores y versiones.

Sobrecargar el servidor sin índices adecuados

Genera lentitud y cuellos de botella.

Falta de pruebas de rendimiento

No validar consultas en escenarios reales puede afectar la experiencia.

No implementar plan de mantenimiento

Como reindexación o limpieza de registros obsoletos.

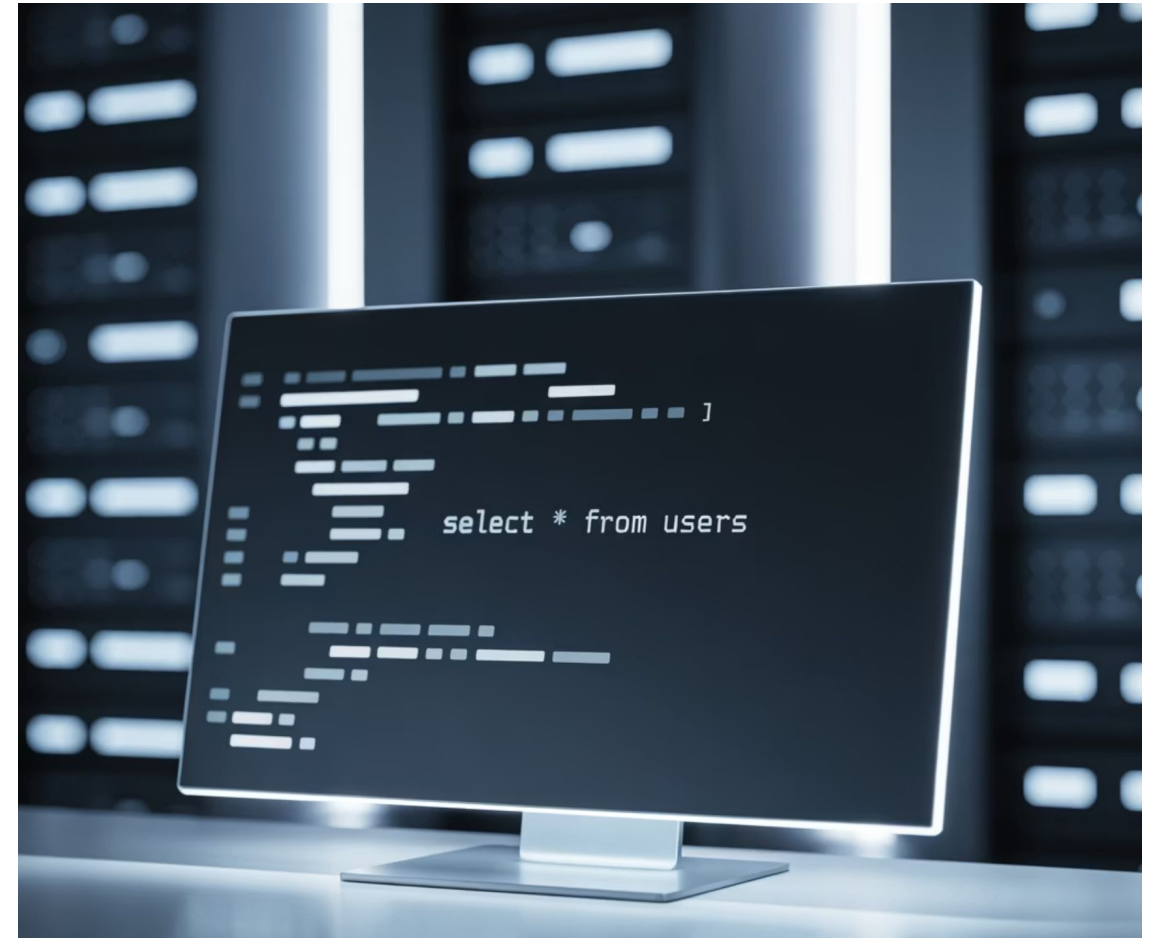
Seleccionar el RDBMS correcto y comprender sus características garantiza un entorno confiable para operaciones y análisis de datos. Ya conociendo el "motor", pasamos a estudiar el "lenguaje" que nos permite interactuar con él: SQL.

¿Qué es SQL y para qué sirve?

¿Cómo podemos interactuar con una base de datos para obtener información específica?

SQL (Structured Query Language) es el lenguaje estándar utilizado para gestionar y consultar bases de datos relacionales. Fue desarrollado por IBM en los años 70 y estandarizado por ANSI y ISO. Su importancia radica en que permite definir estructuras (tablas, vistas), manipular datos (insertar, actualizar, eliminar) y realizar consultas para obtener información relevante.

- ❶ SQL es un lenguaje declarativo, lo que significa que indicamos QUÉ queremos obtener, no CÓMO debe calcularse.



Clasificación de comandos SQL

Antes de operar con SQL, es importante comprender que se divide en varios sublenguajes, cada uno con un propósito específico:



DDL (Data Definition Language)

Crear y modificar estructuras (CREATE TABLE, ALTER TABLE).



DML (Data Manipulation Language)

Operar con datos (INSERT, UPDATE, DELETE).



DQL (Data Query Language)

Consultar datos (SELECT).



DCL (Data Control Language)

Administrar privilegios (GRANT, REVOKE).

Ejemplo:

Una empresa de capacitaciones necesita generar un listado de personas que se inscribieron al curso de "Excel Intermedio".

```
SELECT nombre_estudiante, correo
FROM inscripciones
WHERE curso = 'Excel Intermedio';
```

Esta consulta selecciona dos columnas de la tabla inscripciones, filtrando solo aquellas filas donde el nombre del curso sea 'Excel Intermedio'.

Ventajas de SQL en el trabajo de análisis

SQL (Structured Query Language) no solo es el lenguaje estándar para interactuar con bases de datos relacionales, sino también una herramienta estratégica para quienes trabajan con datos. En contextos productivos reales, las personas analistas deben extraer información precisa desde grandes volúmenes de datos, automatizar reportes, generar alertas o alimentar dashboards. Todo esto se puede lograr usando SQL de forma eficiente.



Acceso directo a datos en su fuente

SQL permite consultar datos desde su origen, sin necesidad de exportarlos a hojas de cálculo o sistemas intermedios. Esto mejora la integridad, reduce errores y agiliza el análisis.



Estandarización y portabilidad

Aprender SQL es una inversión a largo plazo. Su sintaxis es casi idéntica entre distintos motores (PostgreSQL, MySQL, SQL Server), lo que permite moverse entre sistemas sin reaprender desde cero.



Consultas complejas, respuestas rápidas

SQL permite realizar filtros, agregaciones, combinaciones de tablas, subconsultas y cálculos en tiempo real. Con consultas bien escritas e índices adecuados, incluso bases de millones de filas responden en segundos.



Automatización de reportes y procesos analíticos

Las sentencias SQL se pueden integrar en scripts de Python, R o herramientas de visualización como Power BI y Tableau, para crear reportes automáticos o dashboards dinámicos.



Reproducibilidad y auditoría

SQL es transparente. Cada análisis puede ser replicado con exactitud si se conserva la consulta. Esto es esencial para trabajo en equipo, auditorías y trazabilidad.



Independencia técnica

Las personas analistas que dominan SQL no dependen del área de TI para acceder a los datos. Esto reduce tiempos de espera y permite una toma de decisiones más ágil.

Ejemplo:

```
SELECT
    categoria,
    COUNT(*) AS cantidad_ventas,
    SUM(total) AS total_ventas
FROM ventas
WHERE fecha BETWEEN '2024-01-01' AND '2024-06-30'
GROUP BY categoria
ORDER BY total_ventas DESC;
```

Este tipo de consulta podría alimentar directamente un panel de control gerencial mensual sobre comportamiento de ventas por categoría.



Errores frecuentes al aprender o usar SQL en entornos reales

Aunque SQL es intuitivo, su mal uso en entornos reales puede llevar a fallas de seguridad, bajo rendimiento o decisiones basadas en datos incorrectos. Identificar estos errores ayuda a fortalecer el criterio técnico en las personas que inician en el análisis de datos.

Errores comunes:



Omisión de condiciones de filtrado (WHERE)

Ejecutar comandos como DELETE o UPDATE sin una condición puede eliminar o modificar todos los registros. Siempre prueba antes con SELECT para verificar el alcance de la condición.



Abuso del SELECT *

Consultar todas las columnas puede traer datos innecesarios y afectar el rendimiento. Selecciona solo las columnas requeridas, especialmente en sistemas en producción.



Mal uso de los JOIN

Usar INNER JOIN sin considerar registros huérfanos puede hacer que se pierdan datos válidos. Conoce bien la diferencia entre INNER, LEFT, RIGHT y FULL OUTER JOIN.



Olvidar agrupar con GROUP BY

Utilizar funciones como SUM() o COUNT() sin agrupar puede devolver errores o resultados mal interpretados. Siempre acompaña funciones de agregación con un GROUP BY apropiado.



No indexar campos clave

Las consultas sobre tablas grandes sin índices en campos usados por WHERE o JOIN pueden ser lentas e ineficientes. Trabaja con el equipo DBA para definir índices adecuados.



Modificar sin respaldo

Actualizar o eliminar datos en producción sin respaldo puede ser irreversible. Aplica cambios primero en ambiente de pruebas y planifica backups automáticos.

 Ten en cuenta: Antes de ejecutar cualquier sentencia de modificación (UPDATE, DELETE), escribe y prueba primero la lógica usando SELECT.

Comprender las bases de datos relacionales, sus gestores y el lenguaje SQL es una competencia esencial para cualquier persona que trabaje con datos. No se trata solo de almacenar información, sino de garantizar que esté organizada, disponible, segura y lista para responder preguntas de negocio.



Actividad guiada: Estructura relacional para análisis de ventas multicanal

Objetivo de la actividad

Aplicar los conceptos de bases de datos relacionales para identificar entidades, relaciones clave y atributos relevantes dentro de un caso de ventas multicanal.

Ejecutar consultas SQL básicas para recuperar información desde un modelo relacional, simulando requerimientos analíticos de un entorno productivo real.

Situación problema

Eres parte del equipo de análisis comercial de una empresa que vende productos tanto en tiendas físicas como a través de su sitio web. La organización ha implementado un nuevo sistema de base de datos para integrar la información de sus ventas, clientes, productos y sucursales. Actualmente, los reportes de ventas se construyen manualmente en Excel a partir de múltiples fuentes. Esto genera inconsistencias, falta de trazabilidad y tiempos de entrega extensos para los informes semanales que requiere la gerencia comercial. Tu equipo necesita revisar el nuevo modelo de base de datos relacional y realizar algunas consultas iniciales para validar la estructura. Te piden extraer información relevante que permita responder preguntas clave del negocio.

Instrucciones para las personas participantes

Analiza el siguiente modelo de base de datos:

Tablas

clientes

(id_cliente, nombre, correo, ciudad)

productos

(id_producto, nombre_producto, categoría, precio)

sucursales

(id_sucursal, nombre_sucursal, ciudad)

ventas

(id_venta, id_cliente, id_producto, id_sucursal, fecha, cantidad, total)

Ejercicio - Consultas SQL



Instalación PostgreSQL

Para los ejercicios de esta clase utilizaremos herramientas online como DB Fiddle, SQLite Online o SQL Fiddle, que permiten practicar SQL de forma rápida sin necesidad de instalaciones.

Sin embargo, recomendamos instalar [PostgreSQL](#) en tu equipo si deseas profundizar en el uso de bases de datos relacionales y contar con un entorno de trabajo profesional para tus futuros proyectos.

💡 La instalación de PostgreSQL es opcional para este curso, pero te proporcionará:

- Un entorno completo y robusto para practicar SQL
- Experiencia con una herramienta ampliamente utilizada en la industria
- Mayor control sobre tus bases de datos y consultas

📖 Hemos preparado una **guía de apoyo con los pasos detallados para instalar PostgreSQL** que encontrarás disponible en la plataforma como material complementario:

Guía de apoyo - Instalación de PostgreSQL



Insertar datos de ejemplo

Copia y ejecuta el siguiente código SQL para cargar datos iniciales en las tablas:

```
-- Clientes
INSERT INTO clientes VALUES
(1, 'María González', 'maria@email.com', 'Valparaíso'),
(2, 'Carlos Muñoz', 'carlos@email.com', 'Santiago'),
(3, 'Ana Silva', 'ana@email.com', 'Valparaíso');

-- Productos
INSERT INTO productos VALUES
(1, 'Notebook HP', 'Tecnología', 650000),
(2, 'Mouse Logitech', 'Tecnología', 25000);

-- Sucursales
INSERT INTO sucursales VALUES
(1, 'Tienda Valparaíso Centro'),
(2, 'E-commerce Nacional');

-- Ventas
INSERT INTO ventas VALUES
(1, 1, 1, 1, '2024-06-15', 1, 650000),
(2, 3, 2, 2, '2024-06-20', 2, 50000),
(3, 2, 1, 2, '2024-06-25', 1, 650000);
```

Ejercicio - Consultas SQL

(puede hacerse en [DB Fiddle](#), [SQLite online](#) o [SQL Fiddle](#)):

Consulta 1: Obtener los nombres y correos de clientes que viven en Valparaíso.

```
SELECT
    nombre,
    correo
FROM clientes
WHERE ciudad = 'Valparaíso';
```

Consulta 2: Listar las ventas realizadas en el mes de junio, con nombre del cliente y total de la venta.

```
SELECT
    c.nombre,
    v.fecha,
    v.total
FROM ventas v
JOIN clientes c ON v.id_cliente = c.id_cliente
WHERE v.fecha BETWEEN '2024-06-01' AND '2024-06-30';
```



Ejercicio - Consultas SQL

Consulta 3: Mostrar los productos vendidos en cada sucursal junto al nombre de la tienda y la cantidad.

```
SELECT
  s.nombre_sucursal,
  p.nombre_producto,
  SUM(v.cantidad) AS unidades_vendidas
FROM ventas v
JOIN productos p ON v.id_producto = p.id_producto
JOIN sucursales s ON v.id_sucursal = s.id_sucursal
GROUP BY s.nombre_sucursal, p.nombre_producto;
```

Analiza los resultados obtenidos y responde: ¿Qué información permite ver cada consulta? ¿Qué decisión comercial se podría tomar con estos datos?

{desafío}
latam_

Materiales necesarios



Internet y computador con navegador.



Plataforma de ejecución de consultas SQL.



Pizarra o presentación digital para mostrar el modelo relacional.

Actividad práctica:

Construcción y exploración de un modelo relacional de ventas



Objetivo de la actividad

Aplicar de forma autónoma la creación y exploración inicial de una base de datos relacional, simulando un entorno de ventas multicanal.

Esta actividad busca reforzar los conceptos de estructura relacional, llaves primarias y foráneas, y sintaxis SQL básica, utilizando un entorno práctico donde deberás interpretar, consultar y reflexionar sobre el uso estratégico de los datos.



Contexto

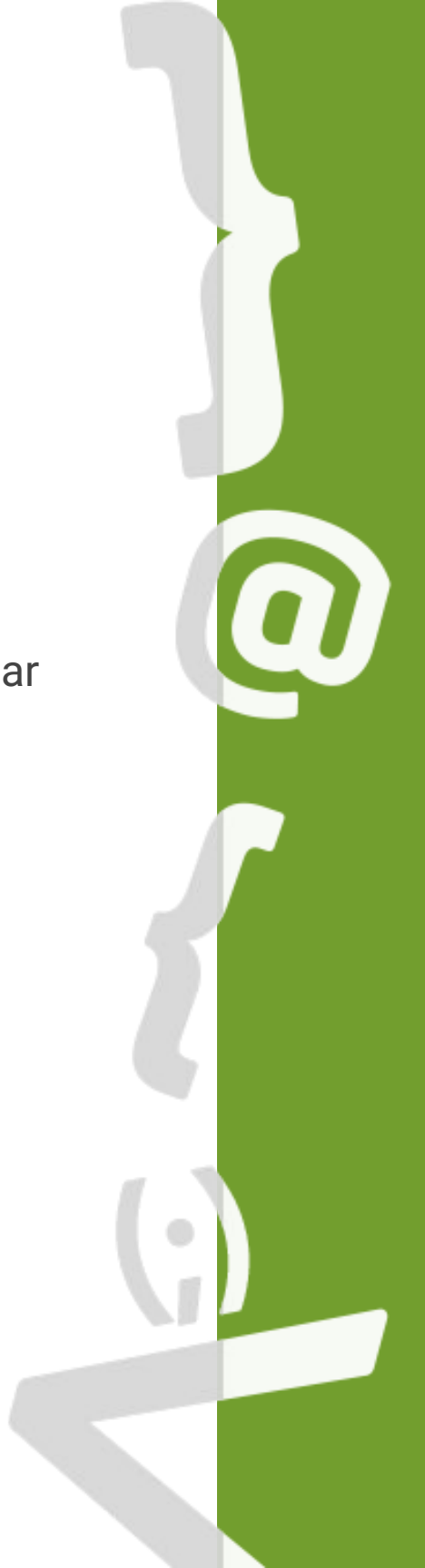
Trabajas en el área de análisis comercial de una empresa de distribución de productos de tecnología, que vende a través de tiendas físicas y canal e-commerce.

El equipo de gerencia requiere visualizar información consolidada de ventas, pero los registros se han llevado por separado en hojas de cálculo de cada sucursal.

Recientemente, se comenzó la implementación de una base de datos relacional centralizada, y te han solicitado probar el modelo y validar que pueda responder preguntas clave del negocio, como:

- ¿Qué productos se han vendido más?
- ¿Qué sucursales generan más ingresos?
- ¿Qué clientes compran con mayor frecuencia?

Para esto, deberás construir un modelo simple y realizar consultas iniciales.



Paso 1: Crear las tablas base

Copia ejecuta el siguiente código SQL en un entorno gratuito como [DB Fiddle](#) o [SQLite Online](#).

```
CREATE TABLE clientes (  
  id_cliente INT PRIMARY KEY,  
  nombre VARCHAR(100),  
  ciudad VARCHAR(50)  
);  
CREATE TABLE productos (  
  id_producto INT PRIMARY KEY,  
  nombre_producto VARCHAR(100),  
  precio DECIMAL(10,2)  
);  
CREATE TABLE sucursales (  
  id_sucursal INT PRIMARY KEY,  
  nombre_sucursal VARCHAR(100)  
);  
CREATE TABLE ventas (  
  id_venta INT PRIMARY KEY,  
  id_cliente INT,  
  id_producto INT,  
  id_sucursal INT,  
  fecha DATE,  
  cantidad INT,  
  total DECIMAL(10,2),  
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),  
  FOREIGN KEY (id_producto) REFERENCES productos(id_producto),  
  FOREIGN KEY (id_sucursal) REFERENCES sucursales(id_sucursal)  
);
```



Paso 2: Insertar algunos datos de ejemplo

```
-- Clientes
INSERT INTO clientes VALUES
    (1, 'Camila Rojas', 'Valparaíso'),
    (2, 'Luis Pérez', 'Santiago');

-- Productos
INSERT INTO productos VALUES
    (1, 'Notebook Lenovo', 550000),
    (2, 'Mouse Inalámbrico', 15000);

-- Sucursales
INSERT INTO sucursales VALUES
    (1, 'Tienda Centro'),
    (2, 'Tienda Online');

-- Ventas
INSERT INTO ventas VALUES
    (1, 1, 1, 2, '2024-06-21', 1, 550000),
    (2, 2, 2, 1, '2024-06-22', 2, 30000);
```



Paso 3: Ejecutar las siguientes consultas

1. Obtener todos los clientes registrados y su ciudad.
2. Consultar todas las ventas realizadas en junio, incluyendo nombre del cliente, producto vendido y sucursal.
3. Calcular el total vendido por producto.

```
-- Consulta 1
SELECT *
FROM clientes;

-- Consulta 2
SELECT
    c.nombre,
    p.nombre_producto,
    s.nombre_sucursal,
    v.total
FROM ventas v
JOIN clientes c ON v.id_cliente = c.id_cliente
JOIN productos p ON v.id_producto = p.id_producto
JOIN sucursales s ON v.id_sucursal = s.id_sucursal
WHERE v.fecha BETWEEN '2024-06-01' AND '2024-06-30';

-- Consulta 3
SELECT
    p.nombre_producto,
    SUM(v.total) AS total_vendido
FROM ventas v
JOIN productos p ON v.id_producto = p.id_producto
GROUP BY p.nombre_producto;
```



Una vez que completes las consultas, completa la siguiente tabla de reflexión en tu cuaderno o cuaderno digital:

Pregunta	Respuesta / Reflexión breve
¿Cuál fue la estructura relacional creada?	
¿Qué tipo de relaciones identificaste?	
¿Qué consultas resultaron más útiles?	
¿Cómo podrían estas consultas ayudar a la gestión comercial?	

Resumen



Concepto de Base de Datos Relacional

Comprendimos el concepto de base de datos relacional y su aplicación en contextos organizacionales reales.



Fundamentos de SQL

Aprendimos los fundamentos del lenguaje SQL y su utilidad para realizar consultas estructuradas y confiables.



Beneficios de RDBMS

Identificamos los beneficios clave de utilizar sistemas gestores de bases de datos (RDBMS) como herramientas para centralizar, asegurar y estructurar datos complejos.



Aplicación Práctica

Aplicamos nuestros conocimientos mediante una actividad guiada sobre un escenario de ventas multicanal y una práctica autónoma de construcción y consulta de datos relacionales.



Motores RDBMS más Utilizados

Conocimos los motores más utilizados en la industria y sus características diferenciales.

Preguntas de cierre

¿Qué características diferencian a una base de datos relacional de una no relacional, y en qué contextos productivos es más adecuada cada una?

¿Cómo se define el concepto de integridad referencial y qué rol cumple en un RDBMS?

¿Cuáles son las principales diferencias entre los motores PostgreSQL, MySQL y SQL Server en términos de casos de uso y licencia?

¿Qué elementos debe contener una consulta SQL para ser segura, eficiente y reutilizable en entornos productivos?

¿Por qué es importante entender la estructura de los datos antes de construir reportes o modelos analíticos?



Próxima sesión...

En la siguiente sesión profundizaremos en los elementos conformantes de una base de datos. Aprenderemos a definir qué es una base de datos desde una perspectiva técnica y operativa, y exploraremos los componentes esenciales que permiten gestionarla eficientemente: usuarios, permisos, tablas, vistas, procedimientos almacenados y funciones.

Esto nos permitirá comprender cómo se organiza internamente un sistema gestor de base de datos y cómo se gestionan los accesos, automatizaciones y vistas controladas en escenarios reales de análisis y operación.

Referencias bibliográficas

- Coronel, C., & Morris, S. (2018). Database systems: Design, implementation, & management (13th ed.). Cengage Learning.
- Oppel, A. (2020). Databases demystified: A self-teaching guide (3rd ed.). McGraw-Hill Education.
- PostgreSQL Global Development Group. (2024). PostgreSQL documentation. <https://www.postgresql.org/docs/>
- W3Schools. (2024). SQL tutorial. <https://www.w3schools.com/sql/>
- Oracle. (2024). What is a relational database?. <https://www.oracle.com/database/what-is-a-relational-database/>
- IBM. (2023). Relational databases explained. <https://www.ibm.com/topics/relational-databases>